

Record-and-Replay of Ground Control Station UIs: Global Event Hooking for Reproducible, Telemetry-Conditioned Scenario Automation

Author Name(s)

Affiliation · e-mail · 2026

Abstract— Ground control stations (GCS) for unmanned aerial vehicles (UAVs) are human-in-the-loop systems whose behaviour is driven through a graphical user interface (UI): an operator arms the vehicle, edits missions, switches modes and triggers guided actions by hand. This manual, UI-mediated nature makes mission-level scenarios difficult to test, demonstrate or reproduce: the same sequence of operations must be re-performed by a person, and time- or state-dependent actions (e.g. "when the vehicle climbs above 500 m, jump to waypoint 3") cannot be expressed at all. We present a framework that turns an unmodified GCS UI into a scriptable, record-and-replay surface. Rather than instrumenting individual widgets, we hook the small set of reusable base controls from which the entire application is composed, so that every button, checkbox, combo box, text field, radio button and slider automatically (i) broadcasts each user interaction as a structured event and (ii) can be actuated from an inbound command exactly as a human click would. On top of this bus we build a scenario engine that supports temporal triggers (relative and absolute time) and *telemetry-conditioned* triggers evaluated against a live vehicle-state stream, enabling an operator to author, save and replay a complete mission start-to-finish. We implement the approach in QGroundControl over a UDP event bus and show that six instrumented base controls cover 225 UI call sites across the application, with ~160 live controls registered at start-up and no per-widget code changes. Round-trip replay reproduces recorded operator sessions, and telemetry conditions fire mission actions autonomously. The result is reproducible, condition-triggered automation of a UAV GCS suitable for regression testing, demonstration and scenario-based evaluation.

Index Terms— ground control station, unmanned aerial vehicle, record-and-replay, UI instrumentation, event hooking, scenario-driven testing, telemetry-conditioned triggers, reproducibility, human-in-the-loop.

I. Introduction

Motivation. A GCS is operated entirely through its UI; therefore testing/demonstrating a UAV mission means a human repeating clicks. This is (a) *non-reproducible* — timing and order vary per run; (b) *non-scriptable* — no way to express state-dependent actions; (c) *labour-intensive* for regression across many vehicle/mission configurations.

Problem statement. How can an existing GCS UI be made *fully recordable and replayable*, and *conditionally scriptable*, without invasive per-widget modification?

Contributions. (1) A *base-control event-hooking* technique that instruments an entire GCS by modifying only the reusable base widgets, giving near-total UI coverage with minimal, non-colliding changes. (2) A bidirectional *event bus* that both broadcasts interactions and replays them as real actuations, with an echo-suppression guard. (3) A *scenario engine* with temporal and telemetry-conditioned triggers, plus a live control-discovery/authoring tool. (4) An implementation in QGroundControl and an empirical characterisation of coverage and replay fidelity.

II. Related Work

GUI record-and-replay and capture/replay testing (coordinate-, widget- and event-level); model-based and scenario-based testing; software/hardware-in-the-loop UAV simulation (PX4 SITL, MAVSDK scripting). Prior GCS automation typically drives the *autopilot* via MAVLink, bypassing the operator UI; we instead automate the *UI itself*, capturing exactly what the human would do, including UI-only state that never reaches the vehicle. [Fill with citations.]

III. Approach

A. Base-control instrumentation

A GCS built on a widget toolkit funnels virtually all interaction through a handful of styled base controls. We attach to each a non-invasive observer (a signal connection that fires *in addition to*, not replacing, the application's own handlers) plus lifecycle registration. Each control derives a stable, human-readable identifier from an explicit id, else its object name, else its type and label, with an ordinal suffix to disambiguate duplicates.

B. Bidirectional event bus

Interactions are emitted on a broadcast channel (`ui`) carrying `{id,type,value}`. An inbound command `{action:"ui",id,value}` looks the control up in a registry and re-emits its native signal, reproducing the exact effect of a user action. A replay guard suppresses re-broadcast during actuation to prevent feedback loops. A discovery command dumps the live control registry for authoring.

C. Scenario engine and conditions

A scenario is an ordered list of steps, each with a trigger and a command. Triggers are `wait` (relative), `at` (absolute), or `when` — a predicate over a periodically-broadcast telemetry stream (`alt_rel`, `groundspeed`, `current_wp`, ...) with a timeout. Commands are any UI id, or high-level actions (e.g. `set-current-waypoint`) that have no single widget. This expresses flows such as "arm → take off → *when alt ≥ 500 m* → go to waypoint 3 → enable search".

IV. Implementation

We implement the framework in **QGroundControl** (Qt/QML). Six base controls are instrumented; a singleton registry mediates broadcast/replay over a UDP bus (Tx 45678, Rx 45679). Telemetry is broadcast every 500 ms per connected vehicle. An external **scenario studio** (Python) provides live control discovery, a telemetry read-out, step authoring, save/load and headless execution.

Base control	Recorded event	Replay actuation
Button	click	emit <code>clicked()</code>
CheckBox / RadioButton	toggle	set <code>checked</code> + <code>clicked()</code>
ComboBox	select	set <code>currentIndex</code> + <code>activated()</code>
TextField	edit	set <code>text</code> + <code>editingFinished()</code>
Slider	move	set <code>value</code> + <code>moved()</code>

Table I. Per-type capture and actuation.

V. Evaluation

We characterise the framework along four axes:

Metric	Definition	Result (this implementation)
Coverage — call sites	UI instances built on the 6 base controls	225 across the source tree
Coverage — live	controls auto-registered at start-up	~160 (no per-widget edits)
Instrumentation cost	files changed to achieve it	6 base-control files + 1 singleton
Replay fidelity	actuation reproduces the real handler	verified via side-channel echo
Condition latency	telemetry-to-action delay	bounded by 500 ms broadcast + poll [measure]

Table II. Evaluation axes and observed results.

Planned experiments: (a) round-trip test — record an operator session, replay, and compare resulting vehicle trajectory / UI state; (b) condition-trigger accuracy across thresholds; (c) coverage ratio vs. total interactive widgets; (d) authoring effort vs. manual MAVLink scripting.

VI. Discussion & Limitations

Identifiers are label-derived and therefore change if UI text changes (mitigated by explicit ids on key controls); a control must be instantiated (its view opened) to be addressable; the approach assumes a toolkit with reusable base controls. It complements, rather than replaces, autopilot-level (MAVLink) scripting by capturing UI-only intent.

VII. Conclusion

By hooking a GCS at its base controls we obtain near-total, low-cost UI instrumentation that makes an otherwise manual, non-reproducible human-in-the-loop system into a recordable, replayable and conditionally scriptable one — enabling reproducible, telemetry-conditioned scenario automation for UAV ground control.

Artifact. Implemented on QGroundControl; scenario studio and manuals are provided with the source. **References.** [To be added.]